PDF BY VP

Telegram Channel

Click here to join

Instagram Page

Click here to join

WhatsApp Channel

Click here to join

✨ Why join?

- Proven notes that helped students ace their exams
- Engineering Quick Updates
- Let's make this Academic year a success together! 🚀

Regards,
PDF BY VP

UNIT 1

Part 1: Programming Paradigms

Before we can appreciate what Object-Oriented Programming (OOP) is, we need to understand what a "programming paradigm" is. Think of it as a **style of building**, a **blueprint**, or a **philosophy** for writing computer programs. It's not a specific language, but a way of thinking about and structuring your code. There are several major paradigms, each with its own strengths.

1.1 The Four Main Programming Paradigms

1. Procedural Programming: The "To-Do List" Paradigm

- **The Big Idea:** This is one of the earliest and most straightforward paradigms. You write a program as a series of sequential, step-by-step instructions, like a recipe or a to-do list. The focus is on **procedures** (also known as functions or subroutines) that perform specific tasks on data. Data and the procedures that operate on it are kept separate.
- **Analogy:** You're building a bookshelf. The procedural approach is a list of instructions: "1. Get wood. 2. Cut wood to size. 3. Get screws. 4. Screw pieces together. 5. Paint the shelf." The data (wood, screws) and the actions (cut, screw, paint) are separate concepts.
- **Characteristics:**
  - Top-down design. ○ Code is organized into functions. ○ Data is often stored in global variables, accessible to all functions.
  - Focuses on the "how" – the sequence of actions.
- **Example Languages:** C, Pascal, FORTRAN.

2. Object-Oriented Programming (OOP): The "World of Things" Paradigm

- **The Big Idea:** Instead of a to-do list, OOP views a program as a collection of interacting **objects**. An object is a self-contained unit that bundles together both **data (attributes)** and the **behaviors (methods)** that operate on that data. This is the cornerstone of OOP.
- **Analogy:** Instead of a list of instructions for building a car, you think about the car *itself* as an object. A Car object has data (color, model, current speed) and

PDF BY VP

behaviors (startEngine(), accelerate(), brake()). The internal complexity of the engine is hidden; you just interact with the simple interface (pedals, steering wheel).

- **Characteristics:**
  - Bottom-up design. ◦ Focuses on "what" things are, not just "how" to do things.
  - Data and behavior are tightly coupled within objects.
  - Emphasizes concepts like Encapsulation, Inheritance, and Polymorphism.
- **Example Languages:** Java, C++, Python, C#.

3. Functional Programming: The "Mathematical Machine" Paradigm

- **The Big Idea:** This paradigm treats computation as the evaluation of mathematical functions. It avoids changing state and mutable data. The core principle is that a function, given the same input, will **always** produce the same output, with no side effects (like modifying a global variable or printing to the screen).
- **Analogy:** It's like a pure mathematical machine. If you put 2 and 3 into an add machine, it will *always* spit out 5. It will never change a number written on a whiteboard outside the machine. This "no side effects" rule is crucial.
- **Characteristics:**
  - Focus on **immutable data** (data that cannot be changed after creation). ◦ Functions are "first-class citizens" (can be passed around like variables).
  - Avoids loops in favor of recursion and higher-order functions (like map, filter, reduce).
- **Example Languages:** Haskell, Lisp, F#, JavaScript (supports functional concepts).

4. Logic & Rule-Based Programming: The "Detective" Paradigm

- **The Big Idea:** This paradigm is completely different. Instead of telling the computer *how* to solve a problem, you tell it *what* you know (a set of facts and rules) and then ask it a question. The computer uses logical deduction to figure out the answer.
- **Analogy:** You are a detective. You have a list of facts ("Socrates is a man.") and rules ("All men are mortal."). You then ask a question ("Is Socrates mortal?"). The logic engine connects the dots and answers "Yes." You didn't tell it the steps to figure it out; you just gave it the knowledge base.
- **Characteristics:**

PDF BY VP

Click Here To Get All Subject

    - Declarative style (you declare what is true, not how to compute). ○ Based on formal logic.
    - Consists of a database of facts and rules.
- **Example Language:** Prolog.

1.2 The Need for Object-Oriented Programming

As programs grew larger and more complex, the procedural paradigm started to show its weaknesses. Global data could be modified by any function anywhere in the program, leading to unpredictable bugs that were hard to trace ("spaghetti code"). It was difficult to model real-world problems and reuse code effectively.

OOP was born out of the need to manage this complexity.

- **Managing Complexity:** By bundling data and behavior into objects, OOP creates self-contained, manageable units.
- **Real-World Modeling:** It's more natural to think about problems in terms of real-world objects (a Student, an Invoice, a Button) rather than abstract procedures.
- **Code Reusability:** Inheritance allows us to create new classes that reuse the properties and methods of existing classes.
- **Maintainability:** Encapsulation protects data from accidental modification, making code safer and easier to maintain and debug.

Part 2: The Core Principles of OOP

These are the fundamental concepts that define the object-oriented paradigm.

- **Class:** The **blueprint** or template for creating objects. A class defines the common attributes (data members) and behaviors (methods) that all objects of its type will have.

```
public class Car {

String color;     int

speed;


   void drive() {
```

PDF BY VP

Click Here To Get All Subject

```
        System.out.println("Car is driving...");
    }
}
```

- **Object:** An **instance** of a class. It is a concrete entity created from the class blueprint, residing in memory. You can create many objects from a single class.

```
public class Main {      public static void
main(String[] args) {
        Car myCar = new Car();  // Object created from Car class
myCar.color = "Red";      myCar.drive();      // Calls the drive
method
    }
}
```

- **Data Members (Attributes/Fields):** The variables inside a class that hold the state or properties of an object. **Data members** are variables defined inside a class. They represent the **state** of an object.

```
public class Student {
String name;    int age;
}
```

- **Methods (Behaviors):** The functions defined inside a class that perform actions or operate on the object's data. **Methods** are functions defined inside a class. They represent the **behavior** of an object.

```
public class Student {    void
study() {
        System.out.println("Student is studying...");
    }
}
```

PDF BY VP

Click Here To Get All Subject

- **Data Encapsulation:** The bundling of data (attributes) and the methods that operate on that data into a single unit (the class). This is the foundational concept of OOP.

  public class Account { private double

  balance; public void deposit(double

  amount) { balance += amount;

  }

  public double getBalance() {

  return balance;

  }

  }

- **Information Hiding & Data Abstraction:** These two concepts work together, enabled by encapsulation.
  - **Information Hiding:** The practice of hiding the internal complexity of an object from the outside world. This is achieved by making data members private and providing controlled access through public methods (getters and setters). This protects the object's internal state.

    public class Bank { private String

    password; public void

    setPassword(String pwd) { // logic to

    validate password password = pwd;

    }}

  - **Data Abstraction:** The process of showing only the essential features of an object to the user and hiding the unnecessary details. The simple interface (public methods) is the abstraction.

    abstract class Shape { abstract void draw(); // Only the method name,

    no implementation

PDF BY VP

Click Here To Get All Subject

```
}
```

- **Inheritance:** The mechanism by which one class (the **child** or **subclass**) can acquire the properties and methods of another class (the **parent** or **superclass**). This supports the **"is-a"** relationship.

```
class Animal {      void
makeSound() {
    System.out.println("Animal sound");
  }
}

class Dog extends Animal {    void
bark() {
    System.out.println("Dog barks");
  }
}
```

- **Polymorphism (from Greek, "many forms"):** The ability of an object, method, or operator to take on many different forms. In OOP, it most often refers to the ability of a single interface (like a method name) to represent different underlying forms (implementations).
  - In Java, this is primarily achieved through **method overriding**. A parent class Animal might have a makeSound() method. A Dog subclass would override it to print "Woof," and a Cat subclass would override it to print "Meow." You can call animal.makeSound() on any animal object, and it will perform the correct action.

```
class Animal {      void
makeSound() {
    System.out.println("Animal sound");
  }
```

PDF BY VP

Click Here To Get All Subject

```
}


class Cat extends Animal {
   @Override          void
makeSound() {
      System.out.println("Meow");
   }
}
```

Benefits of OOP

- **Modularity:** Encapsulation creates self-contained objects, making troubleshooting and collaborative development easier.
- **Reusability:** Inheritance allows for the reuse of code from existing classes.
- **Flexibility:** Polymorphism allows a single action to be performed in different ways.
- **Security:** Information hiding protects internal data from accidental corruption.
- **Maintainability:** Code is easier to understand, maintain, and debug.

Part 3: Java as the Vehicle for OOP

Java is a quintessential object-oriented programming language.

- **Why Java is Strongly OOP:** In Java, virtually everything is an object, and all code must reside within a class. You cannot have a standalone function like in C++.
- **Platform Independence (The JVM):**
  - **JVM (Java Virtual Machine):** The JVM is the magic that makes Java platform-independent ("Write Once, Run Anywhere").
  - **Process:**
    1. You write Java source code (.java file).
    2. The Java compiler compiles it into an intermediate, platform-neutral format called **bytecode** (.class file).
    3. The JVM, which is a program designed for a specific operating system (Windows, macOS, Linux), takes this bytecode and translates it into

PDF BY VP

Click Here To Get All Subject

native machine code that the host computer can execute. ○ **Textual Diagram:**

[YourCode.java] --(Compiler)--> [YourCode.class (Bytecode)] --(JVM)--> [Native Machine Code for Windows/Mac/Linux]

3.1 A Simple Java Program Structure package

com.mycompany.project; // Package statement import

java.util.Scanner; // Import statement

// Class definition public class

MyFirstProgram {

// The main method - the entry point of the program public

static void main(String[] args) {

System.out.println("Hello, World!");

} }

- **Documentation Section:** Comments explaining the code.
- **Package Statement:** Declares a "namespace" to organize classes and avoid naming conflicts.
- **Import Statements:** Imports built-in classes from the Java Class Libraries (e.g., Scanner for user input).
- **Class Definition:** Every Java program must have at least one class.
- **Main Method:** The public static void main(String[] args) method is the special method where program execution begins.

In Java, when you declare a variable, you are essentially creating a named box in the computer's memory. What you can put *inside* that box is determined by its data type. There are two fundamentally different kinds of boxes.

1. Primitive Types:

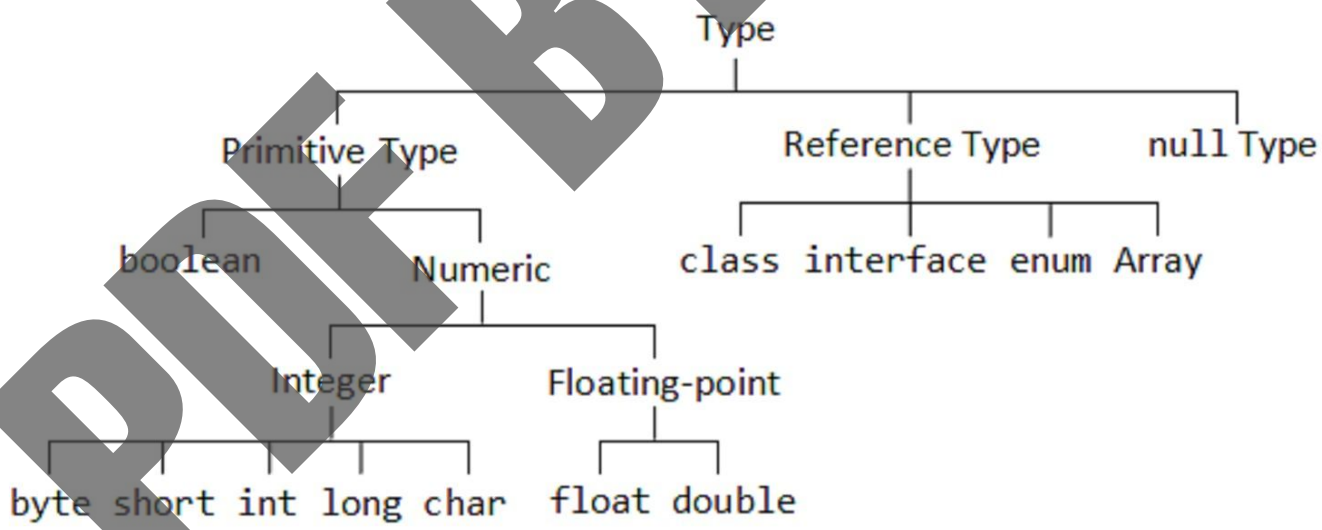
Primitive types are the most basic, fundamental building blocks for data. They are simple and efficient. When you declare a primitive variable, the box in memory **holds the actual value itself**.

PDF BY VP

Click Here To Get All Subject

The 8 Primitive Types in Java:

Type	Size	Description	Example
int	4 bytes	**The default for whole numbers.**	int score = 100;
double	8 bytes	**The default for decimal numbers.**	double price = 19.99;
boolean	1 bit	Stores only true or false.	boolean isLoggedIn = true;
char	2 bytes	Stores a single character (Unicode).	char grade = 'A';
byte	1 byte	For very small whole numbers (128 to 127).	byte smallNumber = 100;
short	2 bytes	For short whole numbers.	short year = 2024;
long	8 bytes	For very large whole numbers.	long population = 8000000000L;
float	4 bytes	For decimal numbers (less precise than double).	float temperature = 98.6f;



PDF BY VP

2. Reference Types:

Reference types are used for more complex data structures and objects. When you declare a reference variable, the box in memory **does not hold the object itself**. Instead, it holds a **memory address**—a "reference" or "pointer"—that tells the computer where the actual, larger object is stored in a different part of memory called the **heap**.

Types of Reference Types:

This includes **all objects** created from classes, and **arrays**.

Click Here To Get All Subject

- **Classes:** String, Scanner, or any class you create yourself (Car, Student, etc.).
- **Arrays:** int[], String[], etc.

Example: Primitive vs. Reference in Action

int x = 10; int y = x; // y gets a COPY of the

value in x y = 20; // We change the value in

y's box.

System.out.println(x); // Output: 10 (x is unaffected)

System.out.println(y); // Output: 20 int[] arr1 = {1, 2,

3};

int[] arr2 = arr1; // arr2 gets a COPY of the ADDRESS in arr1.

// Both variables now point to the SAME array object in memory. arr2[0] =

99; // We go to the address and change the object itself.

System.out.println(arr1[0]); // Output: 99 (arr1 is affected because it points to the same object)

System.out.println(arr2[0]); // Output: 99

PDF BY VP

2. Java Class Libraries - Your Giant Toolkit

The **Java Class Libraries** (also known as the Java API) are a massive collection of prewritten, ready-to-use code that comes with Java. It's like having a giant, professionally organized toolkit. Instead of building everything from scratch, you can just grab a tool that does what you need.

- **Organized into Packages:** These tools are organized into "packages" (like drawers in the toolkit). To use a tool from a package, you must first import it.
- **Example:** If you want to read input from the user, you don't need to write complex code to handle the keyboard. You just import the Scanner class from the java.util package.

import java.util.Scanner; // Importing the Scanner tool from the 'util' drawer.

Click Here To Get All Subject

```
public class UserInputExample {     public

static void main(String[] args) {

      Scanner input = new Scanner(System.in); // Using the tool

      System.out.print("Enter your name: ");

      String name = input.nextLine();

      System.out.println("Hello, " + name);

   }

}
```

3.Control Statements - Directing the Program's Flow

Control statements are the "traffic signals" of your program. They let you make decisions and repeat actions, turning a simple list of instructions into a smart, dynamic program.

1. Selection Statements (Making Decisions)

- **if Statement** ○ **Purpose:** Executes a block of code *only if* a certain condition is true. ○ **Syntax:**

```
if (condition) {

   // Code to execute if condition is true

}
```

  ○ **Example:**

```
int  temperature  =  30;  if

(temperature > 25) {

   System.out.println("It's a hot day!");

}
```

- **if-else Statement** ○ **Purpose:** Provides an alternative path. It executes one block of code if the condition is true, and a *different* block if it is false. ○ **Syntax:**

```
if (condition) {
```

PDF BY VP

Click Here To Get All Subject

```
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

- **Example:**

```
int score = 55; if
(score >= 50) {
    System.out.println("You passed!");
} else {
    System.out.println("You need to study more.");
}
```

- **nested if-else (or if-else if-else)** ○ **Purpose:** Used for a chain of related decisions. It checks multiple conditions in order. ○ **Syntax:**

```
if (condition1) {
    // Code for condition1
} else if (condition2) {
    // Code for condition2
} else {
    // Code if none of the above are true
}
```

- **Example:**

```
int grade = 85; if
(grade >= 90) {
    System.out.println("Excellent (A)");
```

PDF BY VP

Click Here To Get All Subject

```
} else if (grade >= 80) {
    System.out.println("Good (B)");
} else if (grade >= 70) {
    System.out.println("Average (C)");
} else {
    System.out.println("Needs Improvement");
}
```

2. Iteration Statements (Looping)

- **for loop** ◦ **Purpose:** The perfect loop for when you know exactly how many times you want to repeat an action.
  ◦ **Syntax:** for (initialization; condition; update) { ... } ◦ **Example:** Print numbers from 1 to 5.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

- **while loop** ◦ **Purpose:** A flexible loop that repeats as long as a condition is true. The condition is checked *before* each iteration.
  ◦ **Syntax:** while (condition) { ... } ◦ **Example:** A simple countdown.

```
int countdown = 3; while (countdown > 0) {
    System.out.println(countdown);    countdown--; // Don't forget to update the condition variable! }

System.out.println("Blast off!");
```

- **do-while loop** ◦ **Purpose:** Similar to a while loop, but it guarantees that the loop body will be executed **at least once**. The condition is checked *after* each iteration.

PDF BY VP

Click Here To Get All Subject

○ **Syntax:** do { ... } while (condition); ○ **Example:** A menu

that must be shown at least once. Scanner input = new

Scanner(System.in); int choice; do {

System.out.println("--- Menu ---");

System.out.println("1. Start Game");

System.out.println("0. Exit");

System.out.print("Enter your choice: "); choice

= input.nextInt();

} while (choice != 0); // Loop until the user chooses to exit.

- **for-each loop (Enhanced For Loop)** ○ **Purpose:** A clean, simple way to iterate through every element of an array or collection without needing an index variable. ○ **Syntax:** for (DataType variable : collection) { ... } ○ **Example:** Print all names in an array. String[] names = {"Alice", "Bob", "Charlie"}; for (String name : names) {

System.out.println("Hello, " + name);

}

3. Jump Statements

- **break** ○ **Purpose:** To immediately exit the current loop (or switch statement). ○ **Example:** Stop a loop as soon as a target number is found.

int[] numbers = {10, 20, 30, 40, 50};

for (int num : numbers) { if (num

== 30) {

System.out.println("Found 30!");

break; // Exit the loop now.

}

PDF BY VP

Click Here To Get All Subject

```
        System.out.println("Checking " + num);
    }
```

- **continue** ○ **Purpose:** To skip the rest of the current iteration and jump straight to the beginning of the *next* iteration. ○ **Example:** Print only the odd numbers in a range.

```
for (int i = 1; i <= 10; i++) {    if (i % 2 == 0) { // If the number is
even...        continue; // ...skip the print statement and go to the
next 'i'.
  }
  System.out.println(i);
 }
```

Part 5: Case Study - Java's OOP vs. Others

Feature	Java	C++	Python
Paradigm Purity	**Pure OOP.** All code must be inside a class. Primitives are the only exception.	**Hybrid.** Supports both procedural (standalone functions) and object-oriented programming.	**Multi-paradigm & Fully OOP.** Supports procedural, functional, and OOP. Everything, including numbers and functions, is an object.
Inheritance	**Single Inheritance for classes.** A class can only extend one parent class. **Multiple Inheritance** is achieved through **interfaces**.	**Supports Multiple Inheritance.** A class can inherit from several parent classes directly. Can lead to the "Diamond Problem."	**Supports Multiple Inheritance.** Manages it using a Method Resolution Order (MRO) algorithm.

PDF BY VP

Click Here To Get All Subject

Typing System	**Statically Typed.** Types are checked at compile time, leading to more robust and error-free code (but less flexibility).	**Statically Typed.** Similar to Java, provides compile-time safety.	**Dynamically Typed.** Types are checked at runtime ("duck typing"). Offers great flexibility and faster development but can lead to runtime errors.
Memory Management	**Automatic.** Handled by the **Garbage Collector (GC)**. The programmer does not need to manually deallocate memory.	**Manual.** The programmer is responsible for both allocating (new) and deallocating (delete) memory. More powerful but prone to memory leaks.	**Automatic.** Handled by a garbage collector and reference counting.

PDF BY VP

Conclusion: Java enforces a pure OOP structure with a focus on safety and portability (via the JVM and garbage collection). C++ offers more low-level control and performance at the cost of complexity and manual memory management. Python provides maximum flexibility and rapid development speed with its dynamic typing and multi-paradigm approach.

Click Here To Get All Subject

PDF BY VP

Telegram Channel

Click here to join

Instagram Page

Click here to join

WhatsApp Channel

Click here to join

✨ Why join?

- Proven notes that helped students ace their exams
- Engineering Quick Updates
- Let's make this Academic year a success together! 🚀

Regards,
PDF BY VP

UNIT 2 :Introduction to Classes and Objects and Arrays

Classes and Objects in Java

In the last unit, we talked about the *idea* of classes and objects. Now, let's see how we actually build and use them in Java. A class is our blueprint, and an object is the real thing we build from it.

1.1 Defining a Class: Creating the Blueprint

A class is the fundamental building block in Java. It's a container where we define the attributes (data) and behaviors (methods) for a new type of "thing" we want to create.

- **Syntax:**

```
access_modifier class ClassName {
   // --- Fields (Attributes / Data Members) ---
   // These are the variables that hold the state of an object.
   dataType fieldName1;    dataType
fieldName2;

   // --- Methods (Behaviors) ---
   // These are the functions that define what an object can do.
   returnType methodName1(parameters) {
      // Method body (the code)
   }
}
```

- **Field Declaration:** Declaring a field is just like declaring a variable. String name; declares a field to hold a dog's name.
- **Method Declaration & Definition:** A method has a signature (name, parameters, return type) and a body (the code that runs). void bark() { ... } declares a method named bark that takes no parameters and returns nothing.

1.2 Instantiating an Object:

"Instantiating an object" is the fancy term for creating a real object from a class blueprint. This is a two-step process in Java:

1. **Declaration:** You declare a variable of the class type. This creates a "box" that can hold a reference (a memory address).
   Dog myDog; // Creates a box for a Dog reference, but it's currently empty (null).
2. **Instantiation (using the new keyword):** The new keyword is the magic word that tells Java to:
   a. Allocate a chunk of memory on the heap for a new Dog object.
   b. Call the class's **constructor** to initialize this new object.
   c. Return the memory address of the new object.

PDF BY VP

Click Here To Get All Subjects

3. **Assignment:** You assign this returned memory address to your reference variable.
   myDog = new Dog(); // Now, myDog holds the address of the newly created Dog object.

- **Combined in one line (most common):**

// Declare a Dog reference variable and make it point to a new Dog object.
Dog myDog = new Dog();

1.3 Accessing Class Members: Interacting with the Object

Once you have an object, you use the **dot operator (.)** to access its fields and call its methods.

- **Analogy:** The dot is like saying, "Hey, myDog, I want to access your..."
- **Example:**

Dog myDog = new Dog();

// Accessing and setting a field (attribute)
myDog.name = "Buddy";

// Calling a method (behavior)
myDog.bark(); // Output: Buddy says: Woof!

1.4 Methods in Detail: The Workhorses of OOP

- **Declaring Methods with Multiple Parameters:** A method can accept any number of parameters, which are the inputs it needs to do its job.

void setAgeAndBreed(int newAge, String newBreed) { age = newAge;
  breed = newBreed;
}
// Call: myDog.setAgeAndBreed(5, "Golden Retriever");

- **Argument Passing:** In Java, arguments are passed **by value**.
  - **For primitive types (int, double):** A *copy* of the value is passed to the method. If the method changes the parameter, the original variable outside the method is **unaffected**.
  - **For reference types (objects):** A *copy* of the **memory address** is passed. This means both the original variable and the method parameter point to the *exact same object*. If the method uses this reference to change the object's internal state (e.g., theDog.name = "Max"), the change will be visible outside the method.
- **Object as a Parameter:** You can pass entire objects as arguments to methods. This is incredibly powerful for making objects interact.

PDF BY VP

Click Here To Get All Subjects

```
class DogWalker {     void walkDog(Dog theDog) { // Accepts a Dog
object as a parameter
      System.out.println("Walking " + theDog.name);
theDog.wagTail();
   }
}
// Call: DogWalker walker = new DogWalker(); walker.walkDog(myDog);
```

- **Returning Objects:** Methods are not limited to returning simple types like int or boolean. They can create and return entire objects. Generated java class PuppyFactory {

```
   Dog createPuppy(String name) {        Dog puppy = new
Dog();        puppy.name = name;        puppy.age = 0;
return puppy; // Returns a newly created Dog object
   }
}
// Call: Dog fido = puppyFactory.createPuppy("Fido");
```

1.5 Essential Keywords and Concepts

- **Getters and Setters (Accessor and Mutator Methods):** These are public methods used to provide controlled access to private fields, upholding the principle of encapsulation.
  - **Getter:** A method that *gets* (returns) the value of a private field. public String getName() { return this.name; } ○ **Setter:** A method that *sets* (updates) the value of a private field, often with validation logic. public void setName(String name) { this.name = name; }
- **Constructors:** As we've seen, this special method initializes a new object. If you don't provide one, Java gives you a default, no-argument constructor. You can **overload** constructors to provide multiple ways to create an object.

```
class Car {     String
model;
   int year;
   // No-arg constructor
   Car() {
      this.model = "Unknown";
      this.year = 2024;
   }
   // Parameterized constructor
Car(String model, int year) {        this.model
= model;
      this.year = year;
```

PDF BY VP

Click Here To Get All Subjects

```
    }
}
// Car car1 = new Car(); // Uses the no-arg constructor
// Car car2 = new Car("Ford Mustang", 1969); // Uses the parameterized one
```

this Keyword

Purpose:
The this keyword refers to the **current object**. It is used when local variables (like constructor parameters) **have the same names** as instance variables (fields of the class).
Example in Java:

```
class Car {
    String model;
    int year;

    Car(String model, int year) {
        this.model = model; // this.model refers to instance variable
this.year = year;
    }

    void display() {
        System.out.println("Model: " + model + ", Year: " + year);
    }
}
```

Explanation:

- this.model = model; → The first model refers to the class field, the second model is the constructor parameter.
- this removes ambiguity.

Example in C++:

```
class Car {    string
model;
    int year;

public:
    Car(string model, int year) {
        this->model = model; // 'this->' refers to the object's member        this-
>year = year;
    }

    void display() {        cout << "Model: " << model << ", Year: "
<< year << endl;
```

PDF BY VP

Click Here To Get All Subjects

```
    }
};
```

Why use this?
To **differentiate** between instance variables and parameters with the **same name**.

static Members (Fields and Methods) Definition:
A static member belongs to the **class**, not any specific object. So **all objects share the same static variable**.

Example in Java:

```
class Student {
String name;
    static int count = 0; // shared by all Student objects

    Student(String name) {        this.name = name;        count++; //
increment static count when a new student is created
    }

    void display() {
        System.out.println("Name: " + name);
    }

    static void showCount() {
        System.out.println("Total Students: " + count);
    }
}
```

Usage:

- count will be shared among all objects.
- showCount() is a static method, so it can be called like: Student.showCount();.

Example in C++:

```
class
Student {     string
name;
    static int count;

public:
    Student(string n) {
name = n;
        count++;
    }
```

PDF BY VP

Click Here To Get All Subjects

```
    void display() {        cout << "Name: "
<< name << endl;
    }

    static void showCount() {
        cout << "Total Students: " << count << endl;
    }
};

int Student::count = 0; // initialize static variable outside the class
```

Arrays in Java - Organized Collections

An array is a fixed-size, indexed collection of elements of the same type. It's a reference type in Java.

- **Declaring and Creating Arrays:**
    1. **Declaration:** dataType[] arrayName; (e.g., int[] scores;)
    2. **Creation (Instantiation):** arrayName = new dataType[size]; (e.g., scores = new int[10];)
    3. **Combined:** int[] scores = new int[10];
    4. **Declaration with Initialization:** String[] names = {"Alice", "Bob", "Charlie"};
- **Passing Arrays to Methods:** When you pass an array to a method, you are passing a **reference** (the memory address). This means the method can modify the contents of the original array.

```
void doubleScores(int[] scoreArray) {     for (int i = 0; i < scoreArray.length;
i++) {        scoreArray[i] = scoreArray[i] * 2; // This modifies the original
array!
    }
}
```

- **Multidimensional Arrays:** An array of arrays. A 2D array is the most common, representing a grid. // A 3x4 grid of integers int[][] grid = new int[3][4]; grid[0][1] = 99; // Set the value at row 0, column 1

- **Variable-Length Argument Lists (Varargs):** A feature that allows a method to accept a variable number of arguments. It's syntactic sugar for an array.

```
// The ... indicates varargs public static int sum(int... numbers) {     int
total = 0;     for (int num : numbers) { // Inside, 'numbers' is treated as
an array        total += num;
    }
```

PDF BY VP

Click Here To Get All Subjects

```
    return total;
}
// Can be called with any number of arguments:
// sum(1, 2);
// sum(1, 2, 3, 4, 5);
```

- **Command-Line Arguments:** The String[] args parameter in the main method is an array that holds the arguments passed to the program when it's run from the command line.
    - java MyProgram arg1 arg2 -> args will be an array {"arg1", "arg2"}.

Managing Input/Output (I/O)

Java uses a powerful concept called **Streams** to handle all I/O operations. A stream is a sequence of data flowing from a source to a destination.

- **Byte Streams:** Used for reading and writing binary data (8-bit bytes). These are good for any kind of file (images, executables, etc.).
  Examples: FileInputStream, FileOutputStream.

- **Character Streams:** Used for reading and writing text data (16-bit Unicode characters). They automatically handle character encoding. These are best for text files. Examples: FileReader, FileWriter.

- **Predefined Streams:** Java provides three standard streams that are available automatically:
    1. System.in: The standard input stream (usually the keyboard).
    2. System.out: The standard output stream (usually the console/screen).
    3. System.err: The standard error stream (also usually the console).

- **Reading Console Input:** The modern, easy way is to use the Scanner class, which wraps System.in.

```
import java.util.Scanner;
Scanner scanner = new Scanner(System.in);
System.out.print("Enter your age: "); int age =
scanner.nextInt();
```

- **Writing Console Output:** The simplest way is using System.out.println() (which prints and adds a new line) or System.out.print() (which does not).

PDF BY VP

Click Here To Get All Subjects

- **PrintWriter Class:** A more powerful and flexible way to write formatted text to an output stream (like a file or the console). It provides helpful methods like println and printf.

```
// Writing to a file
PrintWriter writer = new PrintWriter("output.txt");
writer.println("Hello, this is line 1."); writer.printf("My score is %d and
my average is %.2f\n", 95, 92.345); writer.close(); // Very important to
close the writer to save the file!
```

Case Study - Interactive Banking System

Let's demonstrate how these concepts come together in a simple banking system.

The Blueprint (BankAccount.java):

This class encapsulates the data and behavior for a single bank account.

```
public class BankAccount {
   // --- Fields (private for encapsulation) ---     private long accountNumber;     private
String ownerName;     private double balance;     private static long nextAccountNumber
= 100001L; // A static field for unique numbers

   // --- Constructor ---
   public BankAccount(String ownerName, double initialDeposit) {
this.ownerName = ownerName;
     this.balance = initialDeposit;
      this.accountNumber = nextAccountNumber;        nextAccountNumber++; //
Increment the static counter for the next account
   }

   // --- Getters (public access to private data) ---
   public long getAccountNumber() { return this.accountNumber; }     public
String getOwnerName() { return this.ownerName; }     public double
getBalance() { return this.balance; }

   // --- Methods (Behaviors) ---     public
void deposit(double amount) {
      if (amount > 0) {
         this.balance += amount;
         System.out.println("Deposit successful. New balance: " + this.balance);
      } else {
```

PDF BY VP

Click Here To Get All Subjects

```
            System.out.println("Invalid deposit amount.");
        }
    }

    public void withdraw(double amount) {        if
(amount > 0 && amount <= this.balance) {
            this.balance -= amount;
            System.out.println("Withdrawal successful. New balance: " + this.balance);
        } else {
            System.out.println("Withdrawal failed. Insufficient funds or invalid amount.");
        }
    }

    // A method to display account details     public
void displayAccountInfo() {
        System.out.println("--------------------");
        System.out.println("Account Number: " + this.accountNumber);
        System.out.println("Owner Name: " + this.ownerName);
        System.out.println("Current Balance: $" + this.balance);        System.out.println("----
--------------------");
    }
}
```

The Bank (Bank.java):
This class uses an **array of BankAccount objects** to manage all the accounts.

```
public class Bank {
    public static void main(String[] args) {
        // Using an array of objects to hold multiple accounts
        BankAccount[] accounts = new BankAccount[3];

        // Creating (instantiating) multiple objects
        accounts[0] = new BankAccount("Alice Smith", 500.00);
accounts[1] = new BankAccount("Bob Johnson", 1200.50);
accounts[2] = new BankAccount("Charlie Brown", 150.75);

        System.out.println("--- Welcome to the Bank ---");

        // Using a for-each loop to interact with all objects in the array
for (BankAccount account : accounts) {
account.displayAccountInfo();
        }
```

PDF BY VP

Click Here To Get All Subjects

```
        // Demonstrate object interaction
        System.out.println("\n--- Performing Transactions ---");
accounts[0].deposit(200);              // Alice deposits
accounts[1].withdraw(400); // Bob withdraws

        System.out.println("\n--- Final Account States ---");
for (BankAccount account : accounts) {
account.displayAccountInfo();
        }
    }
}
```

This case study demonstrates:

- **Class and Object:** The BankAccount class and the three account objects.
- **Encapsulation:** The balance is private and can only be changed via deposit and withdraw.
- **Static Field:** nextAccountNumber is shared by all objects to ensure unique account numbers.
- **Constructors:** Used to create and initialize the accounts.
- **Array of Objects:** The accounts array holds and manages all the BankAccount objects.

PDF BY VP

Click Here To Get All Subjects

PDF BY VP

Telegram Channel

Click here to join

Instagram Page

Click here to join

WhatsApp Channel

Click here to join

✨ Why join?

- Proven notes that helped students ace their exams
- Engineering Quick Updates
- Let's make this Academic year a success together! 🚀

Regards,
PDF BY VP

UNIT 3 : Inheritance and Polymorphism Exception Handling and Multithreading

Part 1: Inheritance - The Idea

Imagine you have a detailed blueprint for a Vehicle. It defines basic properties like speed and color, and basic behaviors like startEngine() and stopEngine(). Now, you want to design a Car. A car *is a* vehicle, so it should have all those same properties and behaviors.

Instead of starting from scratch, you can simply say that your Car blueprint **inherits** from the Vehicle blueprint. You get all the existing features for free and can then add car-specific things like openTrunk() or rollDownWindows(). This is the essence of inheritance: **code reusability** and creating a logical **"is-a"** hierarchy.

1.1 Superclasses and Subclasses

- **Superclass (Parent/Base Class):** The class whose features are being inherited. It's the more general, foundational blueprint. (e.g., Vehicle).
- **Subclass (Child/Derived Class):** The class that inherits from the superclass. It's the more specialized version. (e.g., Car).
- **The extends Keyword:** In Java, you use the extends keyword to establish this relationship. // Superclass class Vehicle { double speed; public void startEngine() { /* ... */ }

}

// Subclass 'Car' inherits from 'Vehicle' class Car

extends Vehicle { int numberOfDoors; // Car-

specific property public void openTrunk() { /*

... */ }

}

Now, a Car object automatically has a speed field and a startEngine() method, even though they aren't explicitly written in the Car class.

1.2 protected Member:

PDF BY VP

Click Here To Get All Notes

Java provides access specifiers to control visibility. protected is designed specifically for inheritance.

- public: Accessible to everyone.
- private: Accessible only within the same class.
- **protected:** Accessible within the same class, by other classes in the **same package**, and by **subclasses** (even if they are in different packages).

1. public

- **Meaning:** The member (class, method, or variable) is accessible from **anywhere in the program**.
- **Where it's accessible:**
  - Inside the same class
  - Inside other classes in the **same package** ◦ Inside classes in **other packages**
- **Use case:** When you want the method/variable/class to be completely open to all other classes.

Example:

public class Car { public String model; // accessible

from anywhere

---

2. private

- **Meaning:** The member is **only accessible within the same class** in which it is defined.
- **Where it's accessible:**
  - Only inside the class itself
  - Not accessible from subclasses or other classes, even in the same package
- **Use case:** To enforce **encapsulation**, hiding internal details and exposing only what's necessary.

public class BankAccount { private double balance; // cannot access

balance from outside this class }

PDF BY VP

Click Here To Get All Notes

3. **protected**

- **Meaning:** The member is accessible within:
  - The same class
  - Other classes in the **same package**
  - Subclasses (**even if** they are in **different packages**)
- **Where it's accessible:**
  - Same class
  - Other classes in same package
  - Subclasses in any package
  - Non-subclasses in different packages
- **Use case:** When you want to allow subclass access while still restricting general access.

Example:

public class Animal { protected String species; // accessible to subclasses, even

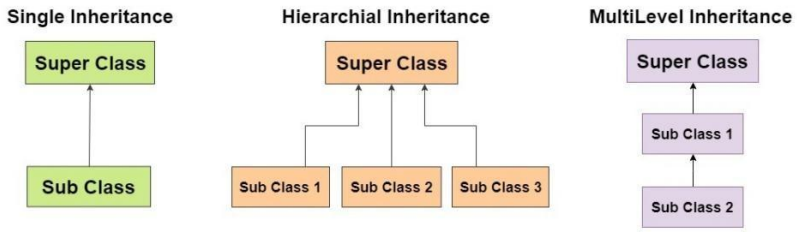
in other packages}

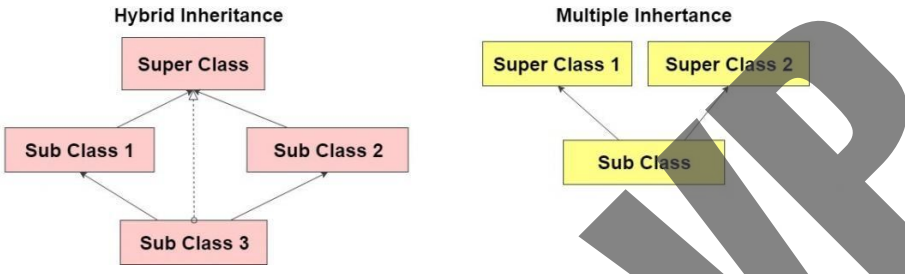
PDF BY VP

1.3 Types of Inheritance (with Textual Diagrams)

1. **Single Inheritance:** One child class inherits from one parent class. (Java supports this directly).
   - A -> B
   -
2. **Multilevel Inheritance:** A class inherits from a parent, which inherits from a grandparent, forming a chain. (Java supports this).
   - A -> B -> C
3. **Hierarchical Inheritance:** One parent class has multiple child classes. (Java supports this).
   - A -> B, A -> C, A -> D
4. **Multiple Inheritance:** One child class inherits from *two or more* parent classes.
   - A -> C, B -> C
   - **Java does NOT support this for classes** because it can lead to the "Diamond Problem" (ambiguity if both parents have a method with the same name). Java solves this using **Interfaces**.
5. **Hybrid Inheritance:** A combination of two or more inheritance types. Not supported directly in Java if it involves multiple inheritance.

Click Here To Get All Notes





1.4 Constructors in Subclasses:

A critical rule of inheritance is that a parent must be fully constructed **before** its child can be constructed. Think about it: you can't build the roof of a house (Car) before the foundation (Vehicle) is in place.

- **The super() Keyword:** A subclass constructor **must** call its superclass's constructor as its very first action.
  - If you don't explicitly write super(), the Java compiler will automatically insert a call to the parent's **no-argument** constructor: super();.
  - If the parent class *only* has a parameterized constructor, you **must** explicitly call it with the required arguments using super(arguments).
- **Example:**

```
class Vehicle {
   String type;
   // Parent's parameterized constructor
public Vehicle(String type) {       this.type
= type;
      System.out.println("Vehicle constructor called.");
   }
}
```

PDF BY VP

Click Here To Get All Notes

```
class Car extends Vehicle {     String
model;
   // Child's constructor     public
Car(String model) {
      // Call the parent's constructor FIRST.
      // This line is mandatory.
super("Four-Wheeler");
this.model = model;
      System.out.println("Car constructor called.");
   }
}

// When you do 'new Car("Ford")', the output is:
// Vehicle constructor called.
// Car constructor called.
```

PDF BY VP

1.5 The Object Class:

In Java, if a class does not explicitly extend another class, it automatically, implicitly extends the **Object** class. This means that **every single class in Java is a descendant of Object**. It is the root of the entire class hierarchy.

Because of this, every object in Java automatically inherits the methods of the Object class, such as:

- toString(): Returns a string representation of the object (which you should override).
- equals(Object obj): Compares two objects for equality (which you should override for meaningful comparison).
- hashCode(): Returns a hash code integer for the object.

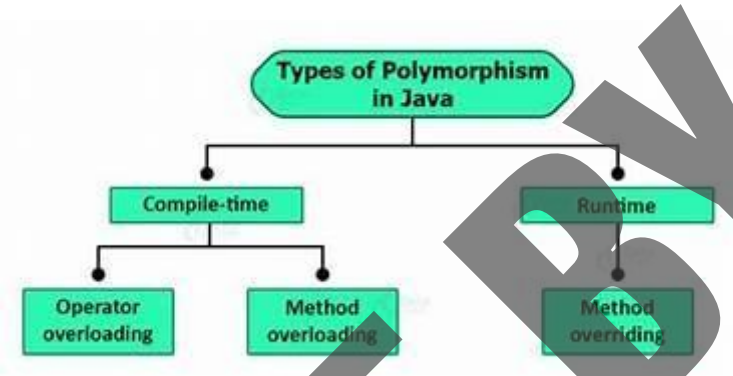
Click Here To Get All Notes

- getClass(): Returns the runtime class of the object.

Part 2: Polymorphism

The Idea:

Polymorphism means "many forms." In OOP, it's the ability to use a single, common interface to represent different underlying actions. Think of the "play" button on a remote. Pressing "play" does one thing for a DVD player, another for a music player, and another for a streaming app. The button (the interface) is the same, but the action (the implementation) is different depending on the object it's controlling.



2.1 Abstract Classes and Methods:

Abstract Class: An abstract class is a class that **cannot be instantiated** (you can't create an object from it with new). It's designed to be a blueprint for other classes to extend. It can contain both regular methods (with bodies) and abstract methods.

- **Abstract Method:** An abstract method is a method that has a signature but **no implementation** (no body, just a semicolon). It acts as a **contract**. Any concrete (non-abstract) subclass that inherits from the abstract class **must** provide an implementation for all its abstract methods.

- **Purpose:** To enforce a common structure and behavior across a group of related subclasses.
- **Example:**

// An incomplete blueprint for any shape. abstract

class Shape {

PDF BY VP

Click Here To Get All Notes

```
    // A regular method, inherited by all children.
    public String getColor() { return "Red"; }

    // An abstract method - a contract.
    // We don't know how to draw a generic shape, but we know all
    // specific shapes MUST be drawable.
    public abstract void draw();
}

class Circle extends Shape {
    // The Circle class MUST implement the draw() method.
    @Override    public
void draw() {
        System.out.println("Drawing a circle O");
    }
}

class Square extends Shape {
    // The Square class MUST also implement draw().
    @Override    public
void draw() {
        System.out.println("Drawing a square []");
    }
```

PDF BY VP

Click Here To Get All Notes

}

2.2 final Methods and Classes:

The final keyword is used to apply restrictions.

- **final Method:** A method declared as final **cannot be overridden** by any subclass.
  - **Purpose:** To ensure that a specific behavior is never changed by a subclass. This is important for security and for methods whose implementation is critical and should not be altered.
- **final Class:** A class declared as final **cannot be extended** (it cannot be a parent class).
  - **Purpose:** To prevent inheritance entirely. The String class in Java is final for security and performance reasons; you cannot create your own version of a String.

2.3 Dynamic Binding:

Dynamic binding (or late binding) is the mechanism that makes runtime polymorphism work. It means that the decision of which specific method implementation to execute is made **at runtime**, based on the **actual type of the object**, not the type of the reference variable pointing to it.

- **Example (using the Shape classes from above):**

// The reference variable is of the parent type 'Shape'.

Shape myShape;

// It points to a Circle object.

myShape = new Circle();

// At RUNTIME, the JVM checks the actual object. It sees a Circle.

// It calls the Circle's version of draw().

myShape.draw(); // Output: Drawing a circle O // Now,

the same reference points to a Square object.

myShape = new Square();

// At RUNTIME, the JVM checks again. It sees a Square.

PDF BY VP

Click Here To Get All Notes

// It calls the Square's version of draw().

myShape.draw(); // Output: Drawing a square []

2.4 Interfaces:

An **interface** is like a **100% abstract class**. It is a reference type that can only contain:

- public static final constants (fields).
- public abstract methods (methods with no body).

A class can **implement** one or more interfaces. When a class implements an interface, it is signing a contract, promising to provide a concrete implementation for **all** the methods defined in that interface.

- **Solving the Multiple Inheritance Problem:** Since a class can implement multiple interfaces, this is how Java achieves the benefits of multiple inheritance without the associated problems. A class can inherit behaviors from many different "contracts."
- **Example:**

// A contract for things that can be flown.

interface Flyable {

void takeOff(); void

fly();

void land();

}

// A contract for things that can be driven.

interface Drivable { void

startEngine(); void

drive();

}

class FlyingCar implements Flyable, Drivable {

PDF BY VP

Click Here To Get All Notes

```
    // This class MUST implement all methods from BOTH interfaces.
    @Override     public void
takeOff() { /* ... */ }
    @Override
    public void fly() { /* ... */ }
    @Override     public void
land() { /* ... */ }
    @Override     public void
startEngine() { /* ... */ }
    @Override     public void
drive() { /* ... */ }
}
```

Part 3: Exception Handling

The Idea:

In a perfect world, our code would always run flawlessly. But in reality, unexpected problems happen: a user enters text instead of a number, a file we need is missing, a network connection drops. An **exception** is an event that disrupts the normal flow of a program's instructions.

Exception handling is a powerful mechanism that allows you to gracefully manage these runtime errors. Instead of letting your program crash, you can "catch" the exception and handle it in a controlled way, allowing the program to recover or terminate cleanl

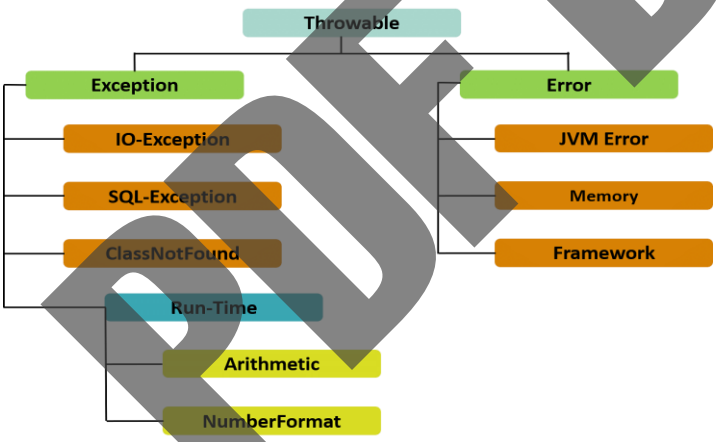
3.1 Fundamentals and Exception Types

In Java, all exceptions are objects that are instances of a class that inherits from the Throwable class.

PDF BY VP

Click Here To Get All Notes

- **The Exception Hierarchy (Simplified):**
  - Throwable
    - Error: Represents severe, system-level problems that your application should not try to catch (e.g., OutOfMemoryError, StackOverflowError). These are usually unrecoverable.
    - Exception: Represents problems that your application *can* and *should* catch and handle.
      - **Checked Exceptions:** These are exceptions that the compiler forces you to handle (either by catching them or declaring that your method throws them). They represent predictable problems, like FileNotFoundException or IOException.
      - **Unchecked Exceptions (Runtime Exceptions):** These are exceptions that you are not required to handle, though you can. They typically represent programming errors, like NullPointerException (trying to use a null reference), ArrayIndexOutOfBoundsException (using an invalid array index), or ArithmeticException (e.g., dividing by zero).



PDF BY VP

Built-in Exceptions in Java

Exception Name	Description
ArithmeticException	Occurs when an illegal arithmetic operation is performed (e.g., divide by zero).
NullPointerException	Thrown when trying to use an object reference that is null.
ArrayIndexOutOfBoundsException	Happens when accessing an array with an invalid index.

Click Here To Get All Notes

NumberFormatException	Occurs when converting a string to a numeric type, and the string is not properly formatted.
ClassCastException	Thrown when trying to cast an object to a subclass it is not an instance of.
IllegalArgumentException	Indicates that a method has been passed an inappropriate or incorrect argument.
StringIndexOutOfBoundsException	Thrown when accessing a character at an invalid index in a string.
FileNotFoundException	Occurs when a file with the specified pathname does not exist (requires java.io).
IOException	General class of exceptions produced by failed or interrupted I/O operations.
InterruptedException	Thrown when a thread is waiting, sleeping, or occupied and is interrupted.
IllegalStateException	Indicates a method has been invoked at an illegal or inappropriate time.
Exception Name	**Description**
InputMismatchException	Occurs when input doesn't match the expected data type (commonly in Scanner).

PDF BY VP

3.2 The try-catch Block :

The try-catch block is the primary tool for handling exceptions.

- **try block:** You place the "risky" code that might throw an exception inside the try block.
- **catch block:** This is the "handler." If an exception of a specific type occurs within the try block, the program immediately jumps to the corresponding catch block to execute the recovery code.
- **Syntax & Example:**

```
try {
```

Click Here To Get All Notes

```
    // Risky code goes here.     int[]
numbers = {1, 2, 3};
    System.out.println("Accessing element 5...");
    System.out.println(numbers[5]); // This will throw an
ArrayIndexOutOfBoundsException
    System.out.println("This line will never be reached.");

} catch (ArrayIndexOutOfBoundsException e) {
    // This block runs ONLY if the specific exception occurs.     System.out.println("Error:
That array index does not exist!");
    // We can print details about the exception object 'e'.
    // e.printStackTrace();
}
System.out.println("The program continues to run after the catch block.");
```

3.3 Handling Multiple Exceptions

A single try block can potentially throw several different types of exceptions. You can handle this in two ways:

- **Multiple catch Clauses:** You can provide a sequence of catch blocks, each one specific to a different exception type. The JVM will match the thrown exception to the first catch block that can handle it.

```
try {
    // Code that might throw different errors
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero.");
```

PDF BY VP

Click Here To Get All Notes

```
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid array index.");
} catch (Exception e) { // A generic catch-all at the end
    System.out.println("An unknown error occurred.");
}
```

- **Nested try Statements:** You can place one try-catch block inside another. This is useful for handling different levels of errors.

3.4 The finally, throw, and throws Keywords

- **finally block:** This is an optional block that is **always executed**, whether an exception was thrown or not. It is the perfect place for cleanup code, such as closing a file or releasing a network connection, to ensure that resources are never left open.

```
// try { ... } catch (Exception e) { ... } finally {
//     // This code runs no matter what.
// }
```

- **throw keyword:** This is used to **manually throw an exception**. You can create a new exception object and throw it yourself to signal that an error condition has occurred in your own logic.

```
public void setAge(int age) {
    if (age < 0) {
        // Manually create and throw an exception.
        throw new IllegalArgumentException("Age cannot be negative.");
    }
    this.age = age;
}
```

PDF BY VP

Click Here To Get All Notes

- **throws keyword:** This is used in a **method signature**. It declares that this method *might* throw a certain type of checked exception, but it is not going to handle it itself. It passes the responsibility of handling the exception up to whatever code calls the method.

```
// This method declares that it might throw an IOException. public

void readFile(String fileName) throws IOException {

   FileReader reader = new FileReader(fileName);

   // ... file reading logic ...

}
```

PDF BY VP

Part 4: Multithreading -

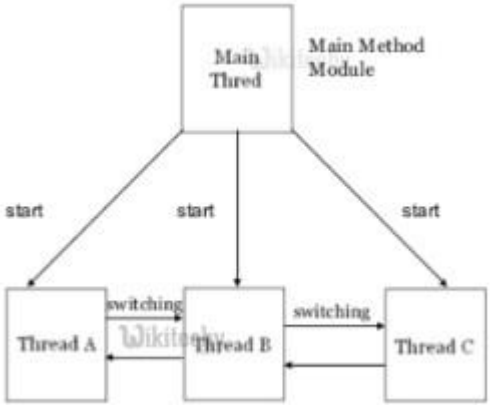
Imagine a restaurant kitchen. A single chef working alone can only cook one dish at a time (this is a **single-threaded** application). To be more efficient, the restaurant hires multiple chefs. Now, one chef can chop vegetables, another can work the grill, and a third can plate the food, all at the same time. This is **multithreading**.

A **thread** is the smallest unit of execution within a program. Multithreading is the ability of a program to run multiple threads concurrently, allowing it to perform several tasks seemingly at once.

4.1 The Java Thread Model and the Main Thread

- **Java Thread Model:** Java has built-in support for multithreading. Each thread has its own path of execution but shares resources like memory with other threads in the same program.
- **The Main Thread:** When you run any Java program, the JVM automatically creates one thread to start with. This is called the **main thread**. It's the thread that executes the main method. Your program ends when the main thread (and all other threads it has created) finishes.

Click Here To Get All Notes



4.2 Creating a Thread

There are two primary ways to create a new thread in Java:

Method 1: By Extending the Thread Class

1. Create a new class that extends Thread.
2. **Override** the run() method in your class. The code inside the run() method is what the new thread will execute.
3. Create an instance (object) of your new class.
4. Call the start() method on the object. The start() method is crucial; it allocates system resources, schedules the thread, and calls your run() method to begin execution.

- **Example:**

```
class MyTask extends Thread {

   @Override    public void

run() {       for (int i = 0; i < 5;

i++) {

         System.out.println("Task 1 is running...");          try {

Thread.sleep(500); } catch (InterruptedException e) {}

      }

   }

}
```

PDF BY VP

Click Here To Get All Notes

Method 2: By Implementing the Runnable Interface (Preferred Method)

1. Create a new class that implements Runnable.
2. Implement the run() method in your class.
3. Create an instance of your Runnable class.
4. Create a new Thread object, passing your Runnable object to its constructor.
5. Call the start() method on the Thread object.

- **Why it's preferred:** Java does not support multiple inheritance for classes. If your class already needs to extend another class, you can't also extend Thread. By implementing Runnable, your class is free to extend something else. It's more flexible.
- **Example:**

```
class AnotherTask implements Runnable {
   @Override
   public void run() {       for
(int i = 0; i < 5; i++) {
         System.out.println("Task 2 is running...");          try {
Thread.sleep(500); } catch (InterruptedException e) {}
      }
   }
}
```

4.3 Creating Multiple Threads

Creating multiple threads is as simple as creating multiple Thread objects and starting them. The Java scheduler will manage running them concurrently.

- **Example putting it all together:**

```
public class MultiThreadDemo {    public
static void main(String[] args) {
      // Create threads using both methods
```

PDF BY VD

Click Here To Get All Notes

```
        MyTask task1 = new MyTask(); // Extends Thread

        Thread task2 = new Thread(new AnotherTask()); // Implements Runnable



        System.out.println("Starting threads...");



        // Start the threads. The order they run in is not guaranteed.

        task1.start();

task2.start();

System.out.println("Main thread

is finished with its tasks.");

    }

}
```

When you run this, you'll see the output from "Task 1," "Task 2," and "Main thread" interleaved, showing that they are all running concurrently.

Part 5: Case Study - Online Banking System

This case study demonstrates how these three advanced concepts can be combined to build a more robust and realistic application.

System Design:

1. **Inheritance for Account Types:**
    - We can have a base **BankAccount** (perhaps abstract) class that defines common properties (accountNumber, balance) and methods (deposit, withdraw).
    - Specialized account types like **SavingsAccount** and **CheckingAccount** can extend BankAccount. They might override the withdraw method to add their own rules (e.g., a SavingsAccount might have a withdrawal limit, while a CheckingAccount might have an overdraft feature).
2. **Exception Handling for Safe Transactions:**

PDF BY VP

Click Here To Get All Notes

- Financial transactions are risky. We need to handle potential errors gracefully.
- The withdraw method should throw a custom **InsufficientFundsException** (a checked exception) if the user tries to withdraw more money than they have.
- The main application code will then wrap all withdrawal calls in a trycatch block to handle this exception and inform the user, instead of letting the program crash or the balance become negative.

3. **Multithreading for Concurrent Operations:**
   - In a real banking system, multiple users (or ATMs) will try to access the same account at the same time. This can lead to data corruption (a "race condition").
   - We can simulate this by creating multiple threads, where each thread represents a user trying to perform a transaction on the same account object.
   - To prevent corruption, the deposit and withdraw methods must be made **thread-safe**. This is typically done by declaring them as **synchronized**, which ensures that only one thread can execute that method on a given object at any one time.

Simplified Code Example: // Custom Exception class

InsufficientFundsException extends Exception { public

InsufficientFundsException(String message) {

super(message);

}

}

// Base Abstract Class using Inheritance

abstract class BankAccount { protected

double balance;

PDF BY VP

Click Here To Get All Notes

```
   // Common methods...     public synchronized void deposit(double amount) {
this.balance += amount; }
   // An abstract contract for withdrawal     public abstract void withdraw(double
amount) throws InsufficientFundsException;
}

// Subclass class SavingsAccount extends
BankAccount {
   @Override
   public synchronized void withdraw(double amount) throws
InsufficientFundsException {       if
(amount > this.balance) {          //
Throw our custom exception
         throw new InsufficientFundsException("Withdrawal amount exceeds balance.");
      }
      this.balance -= amount;
      System.out.println(Thread.currentThread().getName() + " withdrew " + amount + ".
New balance: " + this.balance);
   }
}

// A Runnable task representing a user class UserTransaction implements
Runnable {     private BankAccount account;     public
UserTransaction(BankAccount account) { this.account = account; }
```

PDF BY VP

Click Here To Get All Notes

```
    @Override     public
void run() {        try {

            // Simulate a user trying to withdraw
account.withdraw(100.0);
        } catch (InsufficientFundsException e) {
            // Handle the exception gracefully
            System.out.println(Thread.currentThread().getName() + " failed to withdraw: " +
e.getMessage());
        }
    }
}

// Main class to demonstrate public class
BankingSystemDemo {    public static void
main(String[] args) {
        SavingsAccount sharedAccount = new SavingsAccount();
sharedAccount.deposit(500.0);

        // Create multiple threads (users) trying to access the same account
        Thread user1 = new Thread(new UserTransaction(sharedAccount), "User-Alice");
        Thread user2 = new Thread(new UserTransaction(sharedAccount), "User-Bob");
        Thread user3 = new Thread(new UserTransaction(sharedAccount), "UserCharlie");
```

PDF BY VP

Click Here To Get All Notes

```
        user1.start();
    user2.start();
    user3.start();
  }
}
```

PDF BY VP

Click Here To Get All Notes

PDF BY VP

Telegram Channel

Click here to join

Instagram Page

Click here to join

WhatsApp Channel

Click here to join

✨ Why join?

- Proven notes that helped students ace their exams
- Engineering Quick Updates
- Let's make this Academic year a success together! 🚀

Regards,
PDF BY VP

UNIT 4: Graphics Primitives, Scan Conversion, Windowing and Clipping

The Idea: At its heart, computer graphics is the art of turning mathematical descriptions of shapes (like lines, circles, and polygons) into a grid of colored dots (pixels) on a screen. This entire unit is about that fundamental process: defining the basic building blocks and then figuring out the smartest, fastest ways to "draw" them on a pixel grid.

Part 1: The Basics - Graphics Primitives

Before we can draw anything, we need to understand the canvas and the "paint" we're using.

- **Pixel (Picture Element):** This is the **smallest single dot** on a digital display. It's the most basic, indivisible unit of a digital image. Each pixel can have its own color. A computer screen is just a giant grid of these tiny pixels.
- **Resolution:** This defines how many pixels are in the grid. It's typically expressed as Width x Height (e.g., 1920 x 1080).
    - **Higher resolution** means more pixels, which results in a sharper, more detailed image.
    - **Lower resolution** means fewer, larger pixels, which can make an image look blocky or "pixelated."
- **Aspect Ratio:** The proportional relationship between the width and height of the display. It's expressed as a ratio, like 16:9 (for widescreen displays) or 4:3 (for older, more square displays). Maintaining the correct aspect ratio is crucial to prevent images from looking stretched or squashed.
- **Frame Buffer:** This is a special region of computer memory that holds the color information for **every single pixel** on the screen.
    - **How it works:** The CPU "draws" an image by writing color values into the frame buffer. A video controller then rapidly reads this buffer and sends the information to the monitor to display. To create animation, the CPU quickly updates the frame buffer with new images (frames). ○ The size of the frame buffer depends on the resolution and the color depth (how many bits are used to store the color of one pixel).
- **Display Devices:** The hardware that shows the image, such as CRT (Cathode Ray Tube) monitors (older, bulky), LCD (Liquid Crystal Display) monitors, and OLED (Organic Light-Emitting Diode) displays.
- **Applications of Computer Graphics:**
    - **User Interfaces (GUIs):** Every button, window, and icon you see on your computer or phone.
    - **Entertainment:** Video games, movies (CGI), and animated films.

PDF BY VP

Click Here To Get All Notes

- **Design and Engineering (CAD/CAM):** Computer-Aided Design for creating models of cars, buildings, and products.
- **Scientific Visualization:** Creating 3D models of complex data, like weather patterns, medical scans (MRI, CT), or molecular structures.
- **Simulations and Training:** Flight simulators, surgical training simulators.

Part 2: Scan Conversion - From Math to Pixels

Scan conversion (or rasterization) is the process of converting the mathematical, geometric description of a shape (like a line defined by two endpoints) into a set of discrete pixels that should be turned on to best approximate that shape on the screen.

2.1 Line Drawing Algorithms

The challenge is to figure out which pixels to light up between (x1, y1) and (x2, y2) quickly and accurately, using only integer arithmetic if possible.

PDF BY VP

- **Digital Differential Analyzer (DDA) Algorithm:**
  - **The Idea:** A simple algorithm that calculates the slope of the line and then incrementally steps along the x or y-axis, calculating the corresponding coordinate at each step.
  - **Process:**
    1. Calculate dx = x2 - x1 and dy = y2 - y1.
    2. Determine the number of steps needed (the larger of |dx| or |dy|).
    3. Calculate the increment for each step: x_increment = dx / steps and y_increment = dy / steps.
    4. Start at (x1, y1). Loop for the number of steps, and at each step, plot the pixel (round(x), round(y)) and update x = x + x_increment, y = y + y_increment. ○ **Disadvantage:** It uses **floating-point arithmetic** (/ and +) and rounding, which is computationally slow and can lead to accumulated errors.

Problem: Rasterize a line from point **A(2, 3)** to point **B(8, 7)** using the DDA algorithm.

1. Calculate dx and dy:

- dx = x2 - x1 = 8 - 2 = 6
- dy = y2 - y1 = 7 - 3 = 4

Click Here To Get All Notes

2. Determine the number of steps:

- The number of steps is the larger of |dx| or |dy|.
- |6| = 6, |4| = 4. The larger is 6.
- So, steps = 6.

3. Calculate x_increment and y_increment:

- x_increment = dx / steps = 6 / 6 = 1 • y_increment = dy / steps = 4 / 6 ≈ 0.67

4. Loop and Plot Pixels:

- Start with x = 2 and y = 3.
- We will loop 6 times (from k=0 to 5), plotting (round(x), round(y)) at each step and then incrementing.

Step (k)	Current x	Current y	Plot (round(x), round(y))	New x (x + 1)	New y (y + 0.67)
0	2.00	3.00	**(2, 3)**	3.00	3.67
1	3.00	3.67	**(3, 4)**	4.00	4.34
2	4.00	4.34	**(4, 4)**	5.00	5.01
3	5.00	5.01	**(5, 5)**	6.00	5.68
4	6.00	5.68	**(6, 6)**	7.00	6.35
5	7.00	6.35	**(7, 6)**	8.00	7.02
(end)	8.00	7.02	(Stop after 6 steps)		

Final Pixels Plotted: (2, 3), (3, 4), (4, 4), (5, 5), (6, 6), (7, 6).
(Note: To be precise, the last pixel should align with the endpoint (8,7). Slight rounding variations are common with DDA, which is one of its drawbacks.)

- **Bresenham's Line Algorithm:**
  - **The Idea:** A highly efficient and widely used algorithm that uses **only integer arithmetic** to draw lines. It's a "smart" algorithm that avoids floating-point math entirely.
  - **Core Concept:** At each step along the main axis (e.g., the x-axis), it makes a decision about whether the next pixel should be at the same ycoordinate or one step higher. It does this by keeping track of an **error term** (a decision parameter). The sign of this error term tells it which pixel is closer to the true mathematical line.
  - **Process:**
    1. Initialize a decision parameter based on dx and dy.
    2. Loop along the major axis. At each step:

PDF BY VP

Click Here To Get All Notes

a. Plot the current pixel.
b. Update the decision parameter.
c. If the decision parameter's sign changes, increment the minor axis coordinate.

- **Advantage:** Extremely fast and accurate because it avoids slow floatingpoint operations. This is the standard for line drawing in hardware and software.

Problem: Rasterize a line from A(3, 2) to B(9, 6) using Bresenham's algorithm.

1. Initial Calculations:

- x = 3, y = 2
- dx = x2 - x1 = 9 - 3 = 6
- dy = y2 - y1 = 6 - 2 = 4
- Since dx > dy, the x-axis is the major axis. We will always increment x and decide when to increment y.

2. Calculate the initial Decision Parameter (p):

- p = 2 * dy - dx = 2 * 4 - 6 = 8 - 6 = 2

3. Loop and Plot Pixels:

- We start by plotting the first point: (3, 2).
- We will loop dx = 6 times. At each step, we increment x and check the sign of p.

Iteration	Current p	Action (since p >= 0)	New p	Current x	Current y	Pixel Plotted
						(3, 2)
1	p = 2	p >= 0. Increment y.	p = p + 2dy - 2dx = 2 + 8 - 12 = -2	x=4	y=3	(4, 3)
2	p = -2	p < 0. Don't increment y.	p = p + 2dy = -2 + 8 = 6	x=5	y=3	(5, 3)
3	p = 6	p >= 0. Increment y.	p = p + 2dy - 2dx = 6 + 8 - 12 = 2	x=6	y=4	(6, 4)
4	p = 2	p >= 0. Increment y.	p = p + 2dy - 2dx = 2 + 8 - 12 = -2	x=7	y=5	(7, 5)

PDF BY VP

Click Here To Get All Notes

5	p = -2	p < 0. Don't increment y.	p = p + 2dy = -2 + 8 = 6	x=8	y=5	(8, 5)
6	p = 6	p >= 0. Increment y.	p = p + 2dy - 2dx = 6 + 8 - 12 = 2	x=9	y=6	(9, 6)

Final Pixels Plotted: (3, 2), (4, 3), (5, 3), (6, 4), (7, 5), (8, 5), (9, 6).
All calculations were done using simple integer addition and subtraction, making it very fast.

PDF BY VP

2.2 Circle Drawing Algorithms

The challenge here is to draw a smooth-looking circle using only pixels, again trying to use integer math. We can leverage the circle's eight-way symmetry: if we calculate the pixels for the first octant (from 0 to 45 degrees), we can determine the pixels for the other seven octants by symmetry.

- **Bresenham's Circle Algorithm:**
  - **The Idea:** Similar to his line algorithm, this uses a decision parameter to determine which of two possible pixels is closer to the true circle's edge at each step.
  - **Process:** It starts at (0, r) and steps along the x-axis until x=y. At each x position, it decides whether the next y should be the same or y-1. This decision is based on an integer error term that tracks how far the pixel is from the ideal circle equation ($x^2 + y^2 - r^2 = 0$).

Problem: Draw a circle with its center at **(0, 0)** and a radius of **r = 8**.

1. Initial Calculations:

- Start at the top of the circle: (x, y) = (0, 8).
- Calculate the initial Decision Parameter (d):
  d = 3 - 2 * r = 3 - 2 * 8 = 3 - 16 = -13

2. Loop and Plot Pixels (for the first octant, from 0 to 45 degrees):

- Start by plotting the first point: **(0, 8)**.
- We will loop while x <= y, incrementing x at each step.
  - If d < 0, we choose the **East** pixel (E). This means y stays the same. The update formula is d = d + 4*x + 6.

Click Here To Get All Notes

- If d >= 0, we choose the **South-East** pixel (SE). This means we decrement y. The update formula is d = d + 4*(x-y) + 10.

Iteration	**Current (x,y)**	**Plot (x,y)**	**Current d**	**Action (since d < 0)**	**New d**	**New (x,y)**
		(0 , 8)				
1	(0, 8)		d = -13	d < 0. Choose **E**.	d = d + 4x + 6 = 13 + 4(1) + 6 = **-3**	x=1, y=8
2	(1, 8)	**(1, 8)**	d = -3	d < 0. Choose **E**.	d = d + 4x + 6 = -3 + 4(2) + 6 = **11**	x=2, y=8
3	(2, 8)	**(2, 8)**	d = 11	d >= 0. Choose **SE**.	d = d + 4(x-y) + 10 = 11 + 4(3-7) + 10 = **-5**	x=3, y=7
4	(3, 7)	**(3, 7)**	d = -5	d < 0. Choose **E**.	d = d + 4x + 6 = -5 + 4(4) + 6 = **17**	x=4, y=7
5	(4, 7)	**(4, 7)**	d = 17	d >= 0. Choose **SE**.	d = d + 4(x-y) + 10 = 17 + 4(5-6) + 10 = **15**	x=5, y=6
6	(5, 6)	**(5, 6)**	d = 15	d >= 0. Choose **SE**.	d = d + 4(x-y) + 10 = 15 + 4(6-5) + 10 = **29**	x=6, y=5

- The loop stops here because x is no longer less than or equal to y (6 > 5).

3. Use Symmetry to Find All Other Points:

The pixels we calculated for the first octant are: **(0,8), (1,8), (2,8), (3,7), (4,7), (5,6)**. Now, we use symmetry to find the points in the other 7 octants. For every point (x, y) we found, we can also plot:

- (y, x)
- (y, -x)
- (x, -y)
- (-x, -y)
- (-y, -x)
- (-y, x)
- (-x, y)

PDF BY VP

Click Here To Get All Notes

For example, from the point **(3, 7)**, we get the 7 other symmetric points: **(7,3), (7,-3), (3,-7), (-3,-7), (-7,-3), (-7,3), and (-3,7)**. By doing this for all the calculated points, we can draw the entire circle.

- **Midpoint Circle Algorithm:**
  - **The Idea:** A very popular and efficient algorithm that is conceptually similar to Bresenham's.
  - **Core Concept:** At each step, it calculates the **midpoint** between the two possible next pixels. It then checks if this midpoint is *inside* or *outside* the true circle.
    - If the midpoint is inside, the outer pixel is closer.
    - If the midpoint is outside, the inner pixel is closer.
  - This check is done using a decision parameter derived from the circle equation. Like Bresenham's, it uses only fast **integer arithmetic**.

Problem: Draw a circle with center **(0, 0)** and radius **r = 10**. We only need to calculate the pixels for the first octant (from x=0 to x=y).

1. Initial Calculations:

- Start at (x, y) = (0, 10).
- Calculate the initial Decision Parameter (p): p = 1 - r = 1 - 10 = -9

2. Loop and Plot Pixels (for the first octant):

- Start by plotting the first point: **(0, 10)**.
- We will loop while x < y, incrementing x at each step and deciding whether to decrement y.

Iteration	**Current (x,y)**	**Plot (x,y)**	**Current p**	**Action (since p < 0)**	**New p**	**New (x,y)**
		(0, 10)				
1	(0, 10)		p = -9	p < 0. Keep y same.	p = p + 2x + 3 = -9 + 2(1) + 3 = **-4**	x=1, y=10
2	(1, 10)	**(1, 10)**	p = -4	p < 0. Keep y same.	p = p + 2x + 3 = -4 + 2(2) + 3 = **3**	x=2, y=10

PDF BY VP

Click Here To Get All Notes

3	(2, 10)	**(2, 10)**	p = 3	p >= 0. Decrement y.	p = p + 2x - 2y + 5 = 3 + 2(3) - 2(9) + 5 = **-4**	x=3, y=9
4	(3, 9)	**(3, 9)**	p = -4	p < 0. Keep y same.	p = p + 2x + 3 = -4 + 2(4) + 3 = **7**	x=4, y=9
5	(4, 9)	**(4, 9)**	p = 7	p >= 0. Decrement y.	p = p + 2x - 2y + 5 = 7 + 2(5) - 2(8) + 5 = **6**	x=5, y=8
6	(5, 8)	**(5, 8)**	p = 6	p >= 0. Decrement y.	p = p + 2x - 2y + 5 = 6 + 2(6) - 2(7) + 5 = **9**	x=6, y=7
7	(6, 7)	**(6, 7)**	p = 9	p >= 0. Decrement y.	p = p + 2x - 2y + 5 = 9 + 2(7) - 2(6) + 5 = **16**	x=7, y=6

- The loop stops here because x is no longer less than y (7 > 6).

Final Pixels for the First Octant: (0,10), (1,10), (2,10), (3,9), (4,9), (5,8), (6,7). *You would then use symmetry to plot the pixels for the other 7 octants. For example, since (3,9) is on the circle, so are (9,3), (-3,9), (-9,3), (3,-9), (9,-3), (-3,-9), and (-9,-3).*

Part 3: Polygons and Filling Them In

- **Polygon:** A closed shape made up of a sequence of straight line segments.
- **Types of Polygons:**
  - **Convex:** A polygon where for any two points inside the polygon, the line segment connecting them is also entirely inside. You can think of it as a polygon with no "dents." All interior angles are less than 180 degrees.
  - **Concave:** A polygon that is "dented." It's possible to draw a line segment between two interior points that goes outside the polygon. At least one interior angle is greater than 180 degrees.
  - **Complex:** A polygon where the edges cross over each other (selfintersecting).
- **Inside-Outside Test:** A test to determine if a given point is inside or outside a polygon. A common method is the **Ray Casting Algorithm**: draw a ray (a line) from the point in any fixed direction and count how many times it crosses the polygon's edges.
  - **Odd number of crossings:** The point is **inside**.

PDF BY VP

Click Here To Get All Notes

- **Even number of crossings:** The point is **outside**.

Polygon Filling Algorithms

These algorithms are used to color the interior of a polygon.

- **Flood Fill (Boundary Fill):**
  - **The Idea:** You start with a "seed" pixel *inside* the polygon and recursively "flood" outwards, changing the color of neighboring pixels until you hit the polygon's boundary color.
  - **Process:**
    1. Start with a seed pixel (x, y).
    2. If the pixel's color is not the boundary color and not the fill color, change it to the fill color.
    3. Recursively call the algorithm for its four neighbors (up, down, left, right). ○ **Disadvantage:** Can be slow and consume a lot of stack space due to deep recursion, especially for large, complex shapes.

- **Scan-Line Fill:**
  - **The Idea:** A much more efficient algorithm that fills the polygon one horizontal line (**scan-line**) at a time.
  - **Process:**
    1. For each scan-line that crosses the polygon:
       a. Find all the intersection points where the scan-line crosses the polygon's edges.
       b. Sort these intersection points by their x-coordinate.
       c. Fill in the pixels between each pair of intersection points (e.g., between the 1st and 2nd, the 3rd and 4th, and so on).
  - **Advantage:** Very fast and efficient, avoiding the problems of recursion. This is the standard method used in most graphics systems.

Part 4: Windowing and Clipping - Focusing on What's Important

- **Window:** The area of the world you want to see. It's the "scene" you've defined.
- **Viewport:** The area on the actual display device where the window will be mapped.

PDF BY VP

Click Here To Get All Notes

- **Viewing Transformation:** The process of mapping the coordinates from the window to the viewport.
- **Clipping:** The process of **cutting off** the parts of your geometric shapes that lie **outside** the viewing window. You only want to waste time drawing what the user can actually see.

Clipping Algorithms

- **Cohen-Sutherland Line Clipping Algorithm:**
  - **The Idea:** A fast algorithm that quickly accepts or rejects entire lines. ○ **Process:**
    1. The viewing window is the center of a 3x3 grid. Each of the 9 regions is assigned a 4-bit **outcode**. The center (the window) has outcode 0000. Other regions have bits set for Top, Bottom, Right, Left.
    2. For a line segment with endpoints P1 and P2:
       - **Trivial Accept:** If the outcodes for both P1 and P2 are 0000, the entire line is inside the window. Keep it.
       - **Trivial Reject:** If the logical AND of the two outcodes is *not* 0000, it means both endpoints are in the same outside region (e.g., both are above the window). The line cannot possibly cross the window. Discard it.
       - **Needs Clipping:** If neither of the above is true, the line needs to be clipped. The algorithm finds the intersection point of the line with a window boundary, effectively cutting the line and creating a new, shorter segment to re-test.

Problem: Clip the line segment from **P1(-1, 5)** to **P2(12, 1)** against a window with corners at **(0, 0)** and **(10, 10)**.

1. Define Window Boundaries and Outcodes:

- x_min = 0, y_min = 0
- x_max = 10, y_max = 10
- **Outcode format (4 bits):** Top, Bottom, Right, Left (T B R L) ○ LEFT = 1 (0001), RIGHT = 2 (0010), BOTTOM = 4 (0100), TOP = 8 (1000)

2. Assign Outcodes to Endpoints:

- **For P1(-1, 5):**
  - x = -1 is less than x_min. It's to the **LEFT**.
  - outcode(P1) = 0001
- **For P2(12, 1):**

PDF BY VP

Click Here To Get All Notes

- x = 12 is greater than x_max. It's to the **RIGHT**.
- outcode(P2) = 0010

3. Trivial Accept/Reject Tests:

- **Trivial Accept:** (outcode(P1) == 0000) AND (outcode(P2) == 0000)?
  - 0001 is not 0000. Test fails.
- **Trivial Reject:** (outcode(P1) & outcode(P2)) != 0000?
  - 0001 & 0010 = 0000. The result is zero. Test fails.
- **Conclusion:** The line needs to be clipped.

4. Perform Clipping:

- **Clip P1:** outcode(P1) is 0001 (LEFT). We need to find the intersection with the left edge (x = x_min = 0).
  - Line slope m = (1 - 5) / (12 - (-1)) = -4 / 13.
  - Using the point-slope formula: y - y1 = m * (x - x1) y - 5 = (-4/13) * (0 - (-1)) y - 5 = -4/13 -> y = 5 - 4/13 ≈ 4.69 The new endpoint is **P1' (0, 4.69)**.
- **Re-test with new segment P1'(0, 4.69) to P2(12, 1):**
  - outcode(P1') is now 0000 (it's on the boundary). outcode(P2) is still 0010 (RIGHT). The segment still needs clipping.
- **Clip P2:** outcode(P2) is 0010 (RIGHT). We need to find the intersection with the right edge (x = x_max = 10). y - y1 = m * (x - x1) y - 5 = (-4/13) * (10 - (-1)) y - 5 = (-4/13) * 11 = -44/13 ≈ -3.38 y = 5 - 3.38 = 1.62 The new endpoint is **P2' (10, 1.62)**.

5. Final Check:

- The final line segment is from **P1'(0, 4.69)** to **P2'(10, 1.62)**.
- outcode(P1') = 0000.
- outcode(P2') = 0000.
- This passes the Trivial Accept test.

Final Clipped Line Segment: from **(0, 4.69)** to **(10, 1.62)**.

- **Sutherland-Hodgeman Polygon Clipping Algorithm:**

PDF BY VP

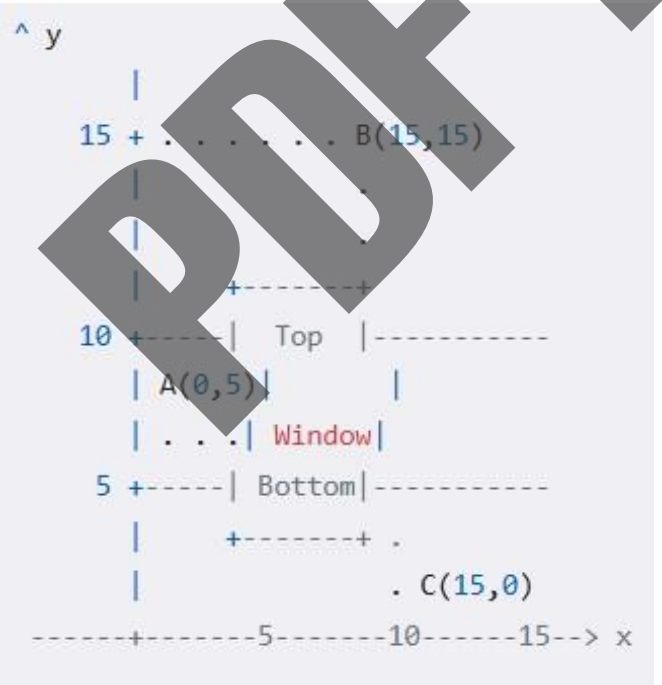
Click Here To Get All Notes

- **The Idea:** Clips an entire polygon against one window edge at a time.
- **Process:**
  1. Take the list of polygon vertices.
  2. Clip this entire list against the **left** edge of the window. This produces a new list of vertices.
  3. Take the new list and clip it against the **right** edge.
  4. Take that result and clip it against the **top** edge.
  5. Finally, take that result and clip it against the **bottom** edge.
- The final list of vertices defines the clipped polygon. The process is a "pipeline" where a polygon is clipped against each of the four infinite boundary lines of the window in sequence.

Problem: Clip the triangle with vertices **A(0, 5)**, **B(15, 15)**, **C(15, 0)** against a window with corners at **(5, 5)** and **(10, 10)**.

Window Boundaries:

- x_min = 5 (Left)
- x_max = 10 (Right)
- y_min = 5 (Bottom)
- y_max = 10 (Top) **Textual Diagram:**



Step 1: Clip against the Left Boundary (x = 5)

We process the edges of the polygon [A, B, C] one by one.

- **Edge A -> B:** (from (0,5) to (15,15)) ○ S (A) is **Outside**. P (B) is **Outside**. (But on opposite sides of the line). We need to find the intersection point I1.

PDF BY VP

Click Here To Get All Notes

- The line equation for A-B is y = (2/3)x + 5. At x=5, y = (10/3)+5 ≈ 8.33.
- I_1 = (5, 8.33)
- **Case (Outside -> Outside):** The rule depends on the specific implementation, but here we can see the edge crosses, so we'd output the intersection point. A simpler rule is Outside->Inside which we will use. Let's rephrase:
- S (A) is **Outside**. P (B) is **Inside**. (No, P is also outside). Let's use the 4 rules properly.
- **The Four Rules (for each edge S->P):**
    1. Inside -> Inside: Output P.
    2. Inside -> Outside: Output Intersection.
    3. Outside -> Outside: Output Nothing (unless it crosses).
    4. Outside -> Inside: Output Intersection, then output P.

Let's re-evaluate Edge A -> B: A is Outside, B is Outside. This edge *crosses* the boundary. Let's find the intersection I_1 (5, 8.33). So we keep I_1.

- **Edge B -> C:** (from (15,15) to (15,0)) ○ S (B) is **Outside**. P (C) is **Outside**. ○ Does this edge cross x=5? No. ○ **Action:** Output nothing.
- **Edge C -> A:** (from (15,0) to (0,5)) ○ S (C) is **Outside**. P (A) is **Outside**. This edge crosses the boundary. ○ Intersection with x=5: y = (-1/3)x + 5. At x=5, y = (-5/3)+5 ≈ 3.33. ○ I_2 = (5, 3.33). Let's keep I_2.

This initial pass is tricky. A more formal way to trace:

- **Edge A(out) -> B(out):** This edge crosses into the valid region at I_1=(5, 8.33) and out at I_3=(10, 11.67).
  Let's restart with the formal "list in, list out" pipeline approach.

Pipeline Restart:
Input Vertex List: [A(0,5), B(15,15), C(15,0)]

1. Left Clipper (x=5)

- **Edge A(out) -> B(out):** Crosses. Find intersection I_1 = (5, 8.33). B is still out.
- **Edge B(out) -> C(out):** Does not cross. Output nothing.
- **Edge C(out) -> A(out):** Crosses. Find intersection I_2 = (5, 3.33). A is still out.
    - This is not yielding a polygon. The Sutherland-Hodgeman rules are more specific. Let's apply them strictly.

Applying the 4 Rules Strictly (Pipeline Restart #2)

Input List: [A(0,5), B(15,15), C(15,0)]

PDF BY VP

Click Here To Get All Notes

Pipeline 1: LEFT CLIP (x >= 5)

- **Edge C -> A:** S(15,0) is IN. P(0,5) is OUT. Rule 2: Output intersection. I_1 = (5, 3.33).
- **Edge A -> B:** S(0,5) is OUT. P(15,15) is IN. Rule 4: Output intersection I_2 = (5, 8.33), then output P B(15,15).
- **Edge B -> C:** S(15,15) is IN. P(15,0) is IN. Rule 1: Output P C(15,0).
- **Output of Left Clipper:** [I_1(5, 3.33), I_2(5, 8.33), B(15,15), C(15,0)]

Pipeline 2: RIGHT CLIP (x <= 10) on the list from above.

- **Edge C -> I_1:** S(15,0) is OUT. P(5, 3.33) is IN. Rule 4: Output intersection I_3 = (10, 1.67), then output P I_1(5, 3.33).
- **Edge I_1 -> I_2:** S(5, 3.33) is IN. P(5, 8.33) is IN. Rule 1: Output P I_2(5, 8.33).
- **Edge I_2 -> B:** S(5, 8.33) is IN. P(15,15) is OUT. Rule 2: Output intersection I_4 = (10, 11.67).
- **Edge B -> C:** S(15,15) is OUT. P(15,0) is OUT. Rule 3: Output nothing.
- **Output of Right Clipper:** [I_3(10, 1.67), I_1(5, 3.33), I_2(5, 8.33), I_4(10, 11.67)]

Pipeline 3: BOTTOM CLIP (y >= 5) on the list from above.

- **Edge I_4 -> I_3:** S(10, 11.67) is IN. P(10, 1.67) is OUT. Rule 2: Output intersection I_5 = (10, 5).
- **Edge I_3 -> I_1:** S(10, 1.67) is OUT. P(5, 3.33) is OUT. Rule 3: Output nothing.
- **Edge I_1 -> I_2:** S(5, 3.33) is OUT. P(5, 8.33) is IN. Rule 4: Output intersection I_6 = (5, 5), then output P I_2(5, 8.33).
- **Edge I_2 -> I_4:** S(5, 8.33) is IN. P(10, 11.67) is IN. Rule 1: Output P I_4(10, 11.67).
- **Output of Bottom Clipper:** [I_5(10, 5), I_6(5, 5), I_2(5, 8.33), I_4(10, 11.67)]

Pipeline 4: TOP CLIP (y <= 10) on the list from above.

- **Edge I_4 -> I_5:** S(10, 11.67) is OUT. P(10, 5) is IN. Rule 4: Output intersection I_7 = (10, 10), then output P I_5(10, 5).
- **Edge I_5 -> I_6:** S(10, 5) is IN. P(5, 5) is IN. Rule 1: Output P I_6(5, 5).
- **Edge I_6 -> I_2:** S(5, 5) is IN. P(5, 8.33) is IN. Rule 1: Output P I_2(5, 8.33).
- **Edge I_2 -> I_4:** S(5, 8.33) is IN. P(10, 11.67) is OUT. Rule 2: Output intersection I_8 = (7.5, 10).
- **Output of Top Clipper (Final Polygon):** [I_7(10, 10), I_5(10, 5), I_6(5, 5), I_2(5, 8.33), I_8(7.5, 10)]

Final Clipped Polygon Vertices:
(10, 10), (10, 5), (5, 5), (5, 8.33), (7.5, 10).
This is the pentagon formed by cutting the original triangle with the window.

PDF BY VP

Click Here To Get All Notes

PDF BY VP

Click Here To Get All Notes

PDF BY VP

Telegram Channel

Click here to join

Instagram Page

Click here to join

WhatsApp Channel

Click here to join

✨ Why join?

- Proven notes that helped students ace their exams
- Engineering Quick Updates
- Let's make this Academic year a success together! 🚀

Regards,
PDF BY VP

UNIT 5: 2D, 3D Transformations and Projections

The Idea: This unit is about manipulating objects in a virtual space. **Transformations** are the mathematical tools we use to move, resize, and rotate objects. **Projections** are the techniques we use to take a 3D scene and create a flat, 2D image of it, just like a camera takes a picture of the real world.

Part 1: 2D Transformations -

2D transformations are operations that change the position, size, or orientation of a 2D object (like a polygon) on a Cartesian plane. We represent objects by their vertices (x, y) and use matrices to perform the transformations.

1.1 Basic 2D Transformations

1. Translation (Moving an Object)

- **Concept:** Translation is the process of moving an object from one position to another without changing its shape, size, or orientation. It's a simple "slide" operation.
- **How it Works:** We add a **translation vector** (tx, ty) to the coordinates of every vertex of the object.
  - tx: The distance to move along the x-axis.
  - ty: The distance to move along the y-axis.
- **Formulas:**
  - x' = x + tx
  - y' = y + ty
- **Matrix Representation (Homogeneous Coordinates):** To represent all transformations as matrix multiplications (which is very efficient for hardware), we use **homogeneous coordinates**. We represent a 2D point (x, y) as a 3x1 vector [x, y, 1].
  - **Translation Matrix (T):**

[1 0 tx]

[0 1 ty]

[0 0 1]

  - **Calculation:** [x', y', 1] = T * [x, y, 1]

2. Scaling (Resizing an Object)

PUT BY VP

Click Here To Get All Subjects

- **Concept:** Scaling changes the size of an object. The object can be enlarged or shrunk.
- **How it Works:** We multiply the coordinates of each vertex by **scaling factors** (sx, sy). The scaling is performed relative to the **origin (0,0)**.
    - sx: Scaling factor along the x-axis. ○ sy: Scaling factor along the y-axis.
    - If sx = sy, it's a **uniform** scaling. If sx != sy, it's a **differential** scaling (causes distortion).
    - If factors are > 1, the object gets bigger. If factors are between 0 and 1, it gets smaller.
- **Formulas:**
    ○ x' = x * sx ○ y'
    = y * sy
- **Scaling Matrix (S):**

[sx 0 0]

[0 sy 0]

[0 0 1]

3. Rotation (Turning an Object)

- **Concept:** Rotation changes the orientation of an object by rotating it around a pivot point. The default pivot is the **origin (0,0)**.
- **How it Works:** We rotate each vertex by an angle θ. By convention, a positive angle θ results in a **counter-clockwise** rotation.
- **Formulas (using standard trigonometry):**
    ○ x' = x * cos(θ) - y * sin(θ) ○ y'
    = x * sin(θ) + y * cos(θ)
- **Rotation Matrix (R):**

[cos(θ) -sin(θ) 0]

[sin(θ) cos(θ) 0]

[0 0 1]

4. Shear (Tilting or Skewing an Object)

- **Concept:** Shear is a transformation that "tilts" or skews an object. The shape is distorted as if its internal layers were slid over each other.
- **How it Works:**

PDF BY VP

Click Here To Get All Subjects

    - **X-Shear:** Shifts x-coordinates based on the y-coordinate. It tilts the object horizontally. x' = x + shx * y.
    - **Y-Shear:** Shifts y-coordinates based on the x-coordinate. It tilts the object vertically. y' = y + shy * x.
- **Shear Matrix (Sh):**
    - *X-Shear:*

[1 shx 0]

[0 1 0]

[0 0 1]

    - *Y-Shear:*

[1 0 0]

[shy 1 0]

[0 0 1]

PDF BY VP

1.2 Composite Transformations: Combining Operations

Often, we need to perform multiple transformations in sequence (e.g., rotate and then move an object). This is done by multiplying the individual transformation matrices together to create a single **composite transformation matrix**.

Important: The order of matrix multiplication matters! T * R is not the same as R * T. Transformations are applied from right to left as they are written.

Rotation about an Arbitrary Point (px, py)

You cannot directly rotate an object around a point other than the origin. You must perform a sequence of three transformations:

1. **Step 1: Translate to Origin:** Move the object so that the pivot point (px, py) is at the origin (0,0). The matrix for this is T(-px, -py).
2. **Step 2: Rotate:** Perform the standard rotation by angle θ around the origin. The matrix is R(θ).

Click Here To Get All Subjects

3. **Step 3: Translate Back:** Move the object back to its original position. The matrix is T(px, py).

- **Composite Matrix:** M = T(px, py) * R(θ) * T(-px, -py)
- To transform a point P, you calculate P' = M * P.

Part 2: 3D Transformations - Manipulating Objects in Space

3D transformations are a direct extension of 2D transformations. We now work with 3D coordinates (x, y, z) and use **4x4 matrices** with homogeneous coordinates [x, y, z, 1].

- **3D Translation:** Add (tx, ty, tz).
  - x' = x + tx, y' = y + ty, z' = z + tz
- **3D Scaling:** Multiply by (sx, sy, sz).
  - x' = x * sx, y' = y * sy, z' = z * sz
- **3D Shear:** Similar to 2D, but can occur along 6 different axes combinations (e.g., shearing x based on y and z).

3D Rotation:
Rotation in 3D is more complex because it occurs around an **axis**. The basic rotations are around the X, Y, or Z axes.

- **Rotation about Z-axis (Rz):** This is identical to 2D rotation, as the zcoordinate does not change.

[cos(θ) -sin(θ) 0 0]

[sin(θ) cos(θ) 0 0]

[0 0 1 0]

[0 0 0 1]

- **Rotation about X-axis (Rx):** The x-coordinate is fixed.

[1 0 0 0]

[0 cos(θ) -sin(θ) 0]

[0 sin(θ) cos(θ) 0]

PDF BY VP

Click Here To Get All Subjects

[0 0 0 1]

- **Rotation about Y-axis (Ry):** The y-coordinate is fixed.

[cos(θ) 0 sin(θ) 0]

[0 1 0 0]

[-sin(θ) 0 cos(θ) 0]

[0 0 0 1]

Rotation about an Arbitrary Axis:
This is a complex, multi-step process similar to rotating about an arbitrary point. It involves a sequence of translations and rotations to align the arbitrary axis with one of the principal axes (e.g., the Z-axis), performing the simple rotation, and then reversing the alignment steps.

Part 3: Projections - Creating a 2D View of a 3D World

Projection is the process of transforming 3D objects from a 3D coordinate system into a 2D coordinate system to be displayed on a flat screen. It's like a camera capturing an image. There are two main families of projection.

3.1 Parallel Projection

- **The Idea:** In parallel projection, the lines of sight (called **projectors**) from the object to the projection plane (the screen) are **parallel** to each other.
- **Key Characteristic:** Parallel lines in the 3D world remain parallel in the 2D projected image. The size of an object does **not** change with its distance from the camera.
- **Use Case:** Used in engineering and architectural drawings (blueprints, CAD) where measurements and proportions must be preserved. It's not realistic, but it's accurate.

Types of Parallel Projection:

- **Orthographic Projection:** The projectors are **perpendicular** to the projection plane. This gives a flat, head-on view of an object, like a front view, top view, or side view.
  - **Subtypes:**
    - **Isometric:** All three principal axes are foreshortened equally. It shows a balanced "corner" view of an object.

PDF BY VP

Click Here To Get All Subjects

- **Diametric:** Two of the three axes are equally foreshortened.
- **Trimetric:** All three axes are foreshortened by different amounts.

- **Oblique Projection:** The projectors are **not perpendicular** to the projection plane. This allows you to see the front of an object head-on (with no distortion) while still seeing the sides and top at an angle.
  - **Subtypes (differ by how the depth is scaled):**
    - **Cavalier:** The full depth of the object is shown. This looks unnaturally long and distorted.
    - **Cabinet:** The depth is scaled down (usually by half) to make the object look more realistic and less distorted. This is a common and popular style

Textual Diagram:

```
3D Object ---> [Parallel Projectors] ---> 2D Projection Plane
(Image)
    O                                       o
   /|\                                     /|\
  / | \                                   / | \
 A--B--C                                 a--b--c
```

3D Object ---> [Parallel Projectors] ---> 2D Projection Plane (Image)

(In parallel projection, the distance from the object to the plane doesn't change the size of the final image abc).

3.2 Perspective Projection

- **The Idea:** This projection mimics how the human eye and a camera work. The projectors are **not parallel**; they all converge at a single point called the **Center of Projection (COP)**, which represents the position of the camera or the eye.
- **Key Characteristics:**
  - **Perspective Foreshortening:** Objects that are farther away appear smaller. Objects that are closer appear larger. This is what creates the illusion of depth and realism.
  - **Vanishing Points:** Parallel lines in the 3D world that are not parallel to the projection plane will appear to converge at a single point on the horizon, called a **vanishing point**.
- **Use Case:** Used in all applications where realism is important: video games, movies, art, and virtual reality.

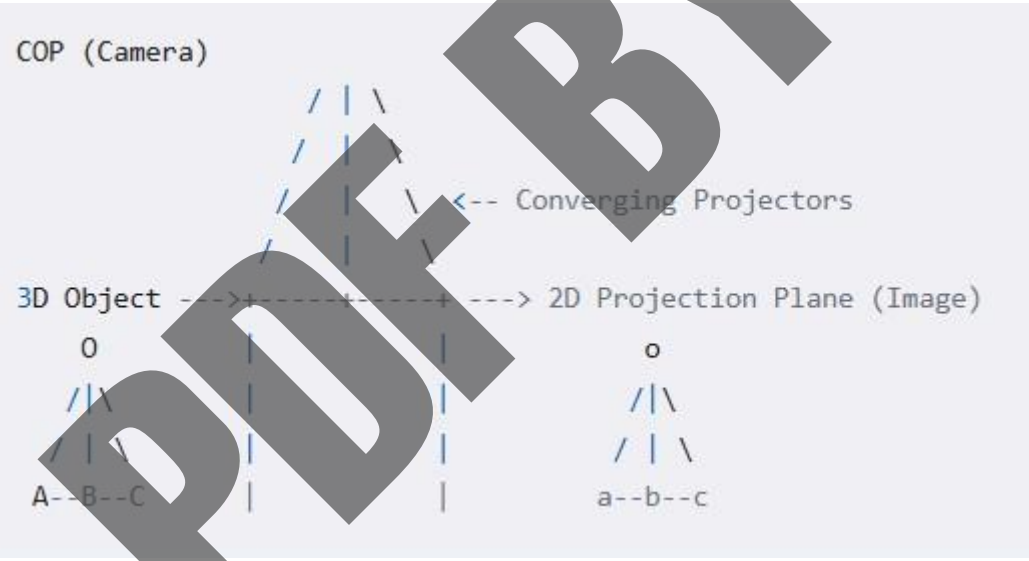
PDF BY VP

Click Here To Get All Subjects

Types of Perspective Projection (classified by the number of vanishing points):

- **One-Point Perspective:** Occurs when the viewer is looking directly at a face of an object, and the parallel lines receding from the viewer converge at a single vanishing point. (e.g., looking straight down a long road or railway track).
- **Two-Point Perspective:** Occurs when the viewer is looking at the corner of an object. There are two sets of parallel lines receding at different angles, so they converge at two separate vanishing points on the horizon. This is a very common view in art and architecture.
- **Three-Point Perspective:** Occurs when the viewer is looking at an object from above (a "bird's-eye view") or below (a "worm's-eye view"). In addition to the two horizontal vanishing points, the vertical lines also appear to converge at a third vanishing point, either below or above the horizon. This creates a dramatic sense of height or depth.

Textual Diagram:



(In perspective projection, if object ABC were further away, the resulting image abc would be smaller.)

PDF BY VP

Click Here To Get All Subjects



CLICK HERE TO JOIN WHATSAPP

CHANNEL FOR SPPU UNIVERSITY

👇**JUST CLICK ON LINK**👇

https://whatsapp.com/channel/0029Va9w1Hg7IUYc